

STANDARD OPERATING PROCEDURE

Access and Security

Who can touch what, and how access is granted and removed

EW-TECH-02 · v1.0 · 2026-09-10

Owner: Brad Palmer · For: Every tech contractor



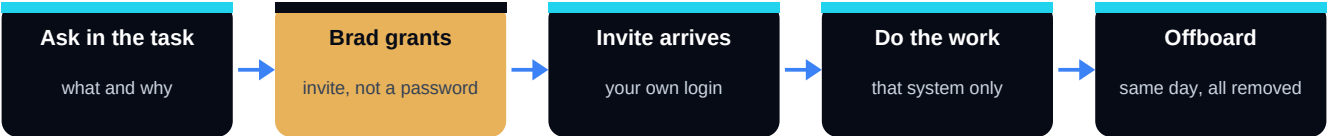
WHAT THIS IS

This document is the access rulebook. It shows who has access to which system, the hard rules that never bend, how you ask for access, how we ask clients for access to their Google and GHL accounts, how secrets are handled in code work, and what happens the day someone leaves. It also tells you exactly what to do if you think a credential has leaked.

DONE MEANS

You only ever work through invites made to your own login, you never see or share a password, and you know the first thing to do when something looks wrong.

01 How access works



Access is requested in a task, granted by Brad as an invite, and removed the day the engagement ends.

Brad	Cash	Technical Builder	Web Developer	Automation Dev
Owner of every system. Grants and removes all access. Holds all secrets.	Head of Sales. Board access on The Bench. Never grants system access.	Client sub account invites plus scoped repo and deploy rights per project.	One repo per client site and deploy rights per site.	Client sub account invites plus scoped API keys per client.

Access at Easyworks is narrow on purpose. Each contractor sees only the client systems their tasks need, through an invite tied to their own login. Nothing is shared, nothing is copied, and everything can be removed in minutes without touching anyone else.

The Bench itself is the first system you get. Your access code gives you the Work board and nothing else. Every other system arrives as an invite after you ask for it in a task.

02 The access matrix

System	Brad	Cash	Technical Builder	Web Developer	Automation Dev
The Bench (easyworks.ai /team/work)	Admin	Approve, assign, send back	Own tasks	Own tasks	Own tasks
GHL agency account	Full	None	Never	Never	Never
GHL client sub accounts	Full	View, if needed	User invite, per client on the task	User invite only if the task needs it	User invite, per client on the task
Client site repos (easyworksai GitHub org)	Full	None	Per project, scoped	Per client site	Per project, scoped
Netlify deploys	All sites	None	Per project	Per site	Per project
easyworks.ai production site	Only Brad	None	Never	Not until trusted, then by Brad	Never

System	Brad	Cash	Technical Builder	Web Developer	Automation Dev
API keys (Claude, ElevenLabs, others)	Full	None	None	None	Scoped key per client, never the master file
Secrets file and shared vault	Full	None	Never	Never	Never
Stripe, client billing, bank	Full	None	Never	Never	Never
Client passwords	Never	Never	Never	Never	Never

CONFIRM WITH BRAD

Cash's day to day access to GHSL sub accounts and the internal Telegram group is not documented in the source facts. The matrix shows the safe reading (view only, when needed). Brad to confirm the exact level.

If a system is not in this table, assume you do not have access and ask in the task. The matrix changes only when Brad updates this document.

03 Hard rules

INVITES, NOT PASSWORDS

Access is always an invite to your own login. Nobody on this team is ever sent a password, and nobody ever sends one. If a password arrives in any channel, stop and follow section 08.

CLIENT SUB ACCOUNTS ONLY

You work in client sub accounts only, through a proper GHSL user invite. The agency account, Stripe, the secrets file, client billing and easyworks.ai production are never in scope for a contractor.

NOTHING SENSITIVE IN CHAT, TASKS, REPOS OR LOOMS

No credentials in chat, tasks, repos or Looms. That includes API keys, tokens, Private Integration Tokens, webhook secrets, and screenshots that show any of them. If it is sensitive it lives in the secrets system Brad runs, nowhere else.

NEVER ASK A CLIENT FOR A PASSWORD

Never ask a client for a password, and never accept one if offered. We ask the client for a manager or viewer invite to our own account. Section 05 shows the invite type for each platform.

WHY

A shared password cannot be removed for one person without changing it for everyone, and you can never prove who used it. An invite can be removed in one click and every action is tied to a name. That is why the rules are absolute.

04 How to request access

1

Ask in the task notes

Say which system, which client, and which SOP step needs it. For example: "Need GHSL user invite to the client sub account for EW-TECH-05 step 2."

If you cannot start without it, press Blocked with the same detail. Brad is pinged either way.

End state: the request is written on the task it belongs to.

2

Brad grants it

Brad sends the invite from the system itself: a GHL user invite, a GitHub repo invite, a Netlify site invite.

Brad records the grant. You do not need to track it.

End state: Brad posts a note saying the invite is sent.

3

It shows up as an invite

Check the email you gave on The Bench, including spam.

Accept it with your own login. Never create a new account with a different email to accept an invite.

End state: you can open the system with your own login and see only what was granted.

4

Confirm in the task

Post a note: "Invite accepted, I can see the sub account."

Then press Start and follow the SOP.

End state: the task moves to In progress with a clean trail of who has what.

TIP

Ask for access when you claim the task, not when you reach the step that needs it. Invites can take a few hours to arrive and be accepted.

05 Client read only access

Most builds need us inside the client's Google and GHL accounts. We always ask the client to invite our team account at the lowest level that does the job. Brad or the onboarding flow makes this ask, usually on Day 0 with the asset request. You never make the ask yourself unless the task says to, and never for a password.

Platform	Invite type we ask for	How the client does it	What it lets us do
Google Business Profile	Manager	Business Profile, Settings, People and access, Add, enter our team email, role Manager.	Edit categories, hours, photos, posts, respond to reviews.
Google Analytics 4	Viewer	Admin, Property access management, Add users, enter our team email, role Viewer.	Read traffic and conversions for the audit and reporting.
Google Search Console	User (restricted or full, Brad decides)	Settings, Users and permissions, Add user, enter our team email.	Read search performance and indexing status.
GoHighLevel (client owned account)	User invite to the sub account	Settings, My Staff, Add employee, enter our team email, role User.	Build inside the sub account without seeing their billing.
Website hosting or domain	Collaborator or delegate access	Depends on the host. Never a login.	Deploy and manage DNS as needed.

CONFIRM WITH BRAD

The exact menu names above are the common paths at the time of writing and may move as Google and GHL change their UI. The rule does not change: manager or viewer invite to our team account, never a password. The team email the client should invite is set by Brad on the task.

WATCH OUT

Verify in the actual dashboard before telling a client what is possible. Do not promise reviews can be moved, listings recovered, or data migrated until you have seen it can be done. A careful "let me confirm" beats a confident wrong answer.

06 Secrets in code work

Web Developers and Automation Devs will handle API keys, Private Integration Tokens, and webhook secrets. The rules below apply to every line of code you ship. See EW-TECH-08 Integrations, Webhooks and AI-Assisted Code for the build patterns.

1

Secrets live in environment variables

Read them with `process.env.NAME` in Node or `os.environ` in Python.

On Netlify they are set in the site's environment settings by Brad, or by you if you hold deploy rights for that site.

End state: your code never contains a literal key.

2

Never commit a secret

Keep a `.env` file only on your machine and make sure `.env` is in `.gitignore` before the first commit.

Before every push, read the diff and search it for the words token, secret and key, and for any long random looking string.

End state: the repo history contains no credential, ever. A committed secret is leaked even if you delete it in the next commit.

3

Never paste a token in a task, chat, or Loom

If you need Brad to check something, describe it. "The PIT for this client returns 401" is enough.

Blur or crop any screenshot before it goes in the notes.

End state: the task trail is safe to read by anyone on the team.

4

Use the scoped key you were given, for that client only

A key for one client never touches another client's code or data.

If a task needs a key you do not have, ask in the task. Brad creates it. Contractors do not create Private Integration Tokens or add to the secrets file.

End state: every key you hold maps to one client and one task.

5

Ask for rotation when in doubt

If a key may have been exposed, even on your own screen share, say so and Brad rotates it. Rotation is cheap.

End state: no key you are unsure about stays live.

AI ASSISTED CODE

AI coding tools will happily write a key straight into a source file or a commit. You are responsible for every line that ships, including the ones the tool wrote. Read the diff before you push.

07 Offboarding, same day

When an engagement ends, for any reason, Brad runs this list the same day. There are no exceptions and no cooling off period. A contractor who is leaving should expect everything below to happen within hours and should not take it personally.

Offboarding checklist (Brad runs this)

- ☐ The Bench: remove the contractor on the Team tab. Their access code stops working immediately.
- ☐ GHL: remove every sub account user invite.
- ☐ GitHub: remove every repo invite in the easyworksai org.
- ☐ Netlify: remove every site deploy right.
- ☐ Rotate anything sensitive they ever saw: API keys, Private Integration Tokens, webhook secrets, shared keys on any site they deployed.
- ☐ Reassign their open tasks on The Bench with a note on where each one stood.
- ☐ Record the removal date in the access log Brad keeps.

IF IT IS YOU

If you are the one leaving: post a final EOD note on every open task, make sure your last work is in the repo or the sub account (not only on your machine), and delete any local copies of client data. Then confirm in a note that you have done so.

08 Security incident: what to do

A credential leaked, or might have. Someone posted a password. A key ended up in a commit. A client account behaves strangely. The response is the same every time: stop, tell Brad immediately, do not try to fix it quietly.

1

Stop

Stop the action you were doing. Do not push, do not deploy, do not send the message.
Do not delete the evidence. A deleted commit or message hides the trail and makes cleanup harder.
End state: nothing new has happened since you noticed.

2

Tell Brad immediately

Message Brad directly, then post in the task. This is one of the two times a direct message is correct.
Say what leaked, where, when, and who could have seen it. Do not include the credential itself in the message.
End state: Brad knows within minutes, not at the next EOD note.

3

Do not fix it quietly

Rotating a key yourself, force pushing over a commit, or asking a client to change a password without Brad's direction can make it worse and breaks the trail.
Brad decides the rotation order, tells the client if needed, and records the incident.
End state: the fix is done once, in the right order, by the person who holds the system.

4

Follow the cleanup Brad assigns

You may be asked to remove a file from history, redeploy with a new key, or re test a client system.
Post proof in the task when each step is done.
End state: the incident task is in Review with the full trail.

RULE

Reporting a leak, including your own mistake, is never held against you. Hiding one is a same day removal. The team is judged on how fast a problem is surfaced, not on whether it happened.

Treat unexpected password reset emails, login alerts and "verify your account" messages as hostile until proven otherwise. Do not click them. Report them in the task and let Brad check the source.

09 Definition of done and when it goes wrong

Definition of done for this SOP

- ☐ Every system I can open, I opened with my own login through an invite.
- ☐ I have not seen, sent, stored or asked for a password on this engagement.
- ☐ No task note, chat, repo or Loom I created contains a key, token or secret.
- ☐ Every key I hold is scoped to one client and was given to me by Brad on a task.
- ☐ My .env is ignored by git and I read the diff before every push.
- ☐ I know the incident rule: stop, tell Brad immediately, do not fix it quietly.

When it goes wrong

Symptom	First three checks	Who to tell
An invite never arrived	1. Spam and promotions folders. 2. The email on My profile matches the one Brad used. 3. It has been at least a few hours.	Note on the task. Brad resends.
I accepted the invite but see nothing, or the wrong client	1. You are logged in with the invited email. 2. You are switched to the right sub account or repo. 3. The invite was to the account, not a different one you own.	Note on the task with a screenshot (no credentials visible).
A client sent me a password	1. Do not use it. 2. Do not store it. 3. Copy nothing, just note that it happened.	Brad, directly and immediately. Brad handles the client and the invite.
I think a key is in a commit or a Loom	1. Stop pushing or sharing. 2. Do not delete or force push. 3. Note where it is.	Brad, directly and immediately, per section 08.
I need a system that is not in the matrix	1. Re read the task, maybe another path exists. 2. Check the SOP for the intended access. 3. Confirm it is really needed for this step.	Press Blocked with the detail. Brad decides.
Someone asks me to share my login or code	1. Refuse. 2. Do not explain the system to them. 3. Note who asked and when.	Brad, directly, same hour.